Alexandria Branch Integration & BGP Troubleshooting Report

Networks Lab — Cairo ↔ Alexandria Topology

---

1. Fixing the SP Cloud (The Physical Link)

The Issue:
BGP between Cairo (R3) and Alexandria (R4) was stuck in the `Active` state, meaning the routers were trying to connect but the pings were failing.

What We Did:
We deleted the GNS3 "Cloud" node completely. In its place, we added a standard Ethernet Switch, connected R3 and R4 to it, and changed its icon to look like a cloud.

Why:
Because both branches were running on the exact same laptop, the GNS3 Cloud node was trying to push traffic out to the laptop's actual physical network adapters (Wi-Fi/Ethernet) instead of keeping it inside the simulation. By replacing it with a switch, R3 and R4 were finally placed on the exact same virtual wire (`10.1.3.0/30`), allowing the BGP handshake to complete.

2. Dropping the BGP Security Shield

The Issue:
Even after BGP established, Alexandria (R4) was refusing to accept Cairo's networks. The `show ip bgp summary` command displayed `(Policy)` under the received routes column.

What We Did:
We ran `no bgp ebgp-requires-policy` inside the BGP 200 configuration on R4.

Why:
FRRouting has a strict default security feature that automatically drops any external BGP routes unless you explicitly permit them. This command dropped the shield, allowing Alexandria to see Cairo's Sales, HR, and IT networks.

3. Kicking the BGP Process

The Issue:
After dropping the policy, the routing table still didn't update.

What We Did:
We ran `clear ip bgp *` on both R3 and R4.

Why:
BGP is notoriously "lazy." It prioritizes stability and will not automatically resend its routing tables just because you changed a configuration setting. The `clear` command temporarily drops the session and forces both routers to immediately exchange their updated route lists.

4. The "Connected Route" Trap

The Issue:
R3 (Cairo) was successfully sending 3 networks, but R4 (Alexandria) was sending `0` networks back. R4 was refusing to share the `50.1.1.0/24` network over BGP.

What We Did:
Instead of relying on redistribution, we went into R4's BGP configuration and manually typed `network 50.1.1.0/24`.

Why:
Because the Alexandria PCs were plugged directly into R4's `eth0` port, the router classified the `50.1.1.0/24` network as a "Connected" (`C`) route. When BGP was told to `redistribute eigrp`, it completely ignored the `50` network because it wasn't labeled with an EIGRP (`E`) tag. Manually advertising the network bypassed this technicality and forced R4 to send it to Cairo.

5. Bridging the Local Protocols

The Issue:
The routers knew the external routes, but the internal PCs didn't.

What We Did:
On R4, we configured `redistribute eigrp` inside the BGP process, and `redistribute bgp` inside the EIGRP process.

Why:
EIGRP (the internal protocol) and BGP (the external protocol) speak completely different languages and refuse to share maps with each other. A two-way bridge was built to translate Cairo's incoming BGP routes into EIGRP so the Alexandria PCs could understand them, and vice versa.

6. Fixing the "One-Way Routing" Bug (Cairo Stub Routers)

The Issue:
Pings from Alexandria to Cairo resulted in a "Timeout", while pings from Cairo to Alexandria resulted in "Destination Unreachable". PC6 and PC7 still couldn't talk.

What We Did:
We configured a Default Route (`ip route 0.0.0.0/0 10.1.1.2` and `10.1.2.2`) on Router 1 and Router 2 in Cairo, pointing them at Router 3.

Why:
This proved that the packets were making it across the cloud, but getting lost on the return trip. R3 knew where Alexandria was, but R1 and R2 were completely blind to it. Because R1 and R2 are "stub" routers (they only have one exit path), they don't need to memorize the entire Alexandria routing table. The default route simply tells them: "If you don't know where a packet goes, hand it to R3." R3 then handles the rest, achieving full end-to-end convergence.

End of Report